

Got a problem statement for which we need to build a website for real-life usecase that can be easily used by non technical users, handles realistic values and scenarios & could be easily deployed and without major changes (it's actually for a promptathon). For now, do not generate the code, just think and evaluate about all the needs, requirements, architecture, design, ui, ux, frontend, backend, data management, security, monitoring etc. Specially consider that it should work flawlessly, user-friendly & also keep the judges pov in mind that what should they judge for or look at and accordingly focus on

Show more ▾



This is actually a very interesting Promptathon problem because **the required features are simple**, but the **judging differentiation will come from how realistically and intelligently you handle money management**.

The biggest mistake many teams will make is building a CRUD expense tracker with a progress bar and then adding the velocity formula. That may satisfy requirements, but it won't impress judges.

1. First Understand The Real Problem

The problem is NOT:

"Store expenses."

The problem is:

"Help a first-year college student avoid running out of money before month end."

That mindset should drive every design decision.



Core User Persona

Primary User

First-year college student

Characteristics:

- Non-technical
- Uses phone more than laptop
- Gets fixed pocket money/stipend
- Doesn't understand budgeting deeply
- Wants quick entry
- Often forgets expenses
- Wants warnings before disaster happens

Therefore:

✗ Complex finance dashboard

✗ Accounting terminology

✗ Multi-level reports

✓ Fast expense logging

✓ Visual understanding

✓ Actionable suggestions

✓ Mobile-first design

2. Requirement Breakdown

Functional Requirements

Budget Setup

User enters:

- Monthly Budget Amount

Example:

₹5000

Store:

budget
month
creation date



Expense Logging

Required categories:

- Food
- Entertainment
- Books/Stationery

Each expense:

id
date
amount
category
timestamp

Spending History

Display:

Date	Category	Amount
12 Jun	Food	₹120

Must support:

- sorting
- filtering
- deletion

Even if not required.

Because judges will try mistakes.

Budget Indicator

Need:

Spent
Remaining
Percentage Used



Visual:

Progress Bar

Example:



Color logic:

Green = Safe

Yellow = Caution

Red = Danger

Velocity Alert

Mandatory twist.

Need:

```
velocity =  
totalSpent / daysElapsed
```

Then calculate:

```
projectedMonthSpend =  
velocity × daysInMonth
```

If `projectedMonthSpend > budget`

trigger warning

3. Going Beyond Requirement (Judge-Friendly Enhancements)

These are the things that make judges say:

"This team thought like product builders."



Smart Daily Limit

Not just:

"Budget will run out"

Also:

Remaining Budget
÷ Remaining Days

Show:

Recommended daily spending:
₹117/day

This is extremely useful.

Safe / Risk Status

Simple status card.

Example:

 Safe

 Watch Spending

 Budget Risk

Much easier than making user interpret numbers.

Monthly Projection Card

Show:

Expected End-of-Month Spend:
₹6200

Budget:
₹5000

Expected Overspend:
₹1200



Judges love predictive features.

4. Data Model

Keep it modular.

Budget Object

JavaScript

```
{  
  budgetAmount: 5000,  
  month: "2026-06"  
}
```

Expense Object

JavaScript

```
{  
  id: "uuid",  
  amount: 120,  
  category: "Food",  
  date: "2026-06-12"  
}
```

Analytics Object

Derived.

Never stored.

JavaScript

```
{  
  totalSpent,  
  remainingBudget,  
  spendingVelocity,  
  projectedSpend,  
  recommendedDailyLimit  
}
```

Derived data prevents inconsistencies.



5. Architecture

For Promptathon:

Layer 1

UI Components

```
BudgetSetup
ExpenseForm
ExpenseHistory
BudgetIndicator
VelocityAlert
AnalyticsCards
```

Layer 2

Business Logic

```
Budget Service
Expense Service
Analytics Service
```

Layer 3

Storage Layer

```
LocalStorage
```

Simple.

No backend required.

Deployment becomes trivial.



Recommended Structure

```
src/
```

```
components/
```

```

    BudgetForm
    ExpenseForm
    ExpenseTable
    ProgressBar
    AlertCard
    AnalyticsCards

services/
  budgetService
  expenseService
  analyticsService

utils/
  dateUtils
  validationUtils
  calculationUtils

storage/
  localStorageManager

styles/
```

This screams modular design.

6. UX Flow

First Visit

Budget setup screen.

```

Enter Monthly Budget

[ ₹5000 ]

[ Start Tracking ]
```

Dashboard

Top:

Budget Overview

Middle:

Progress Bar



Analytics

Bottom:

Expense Entry

History

Flow should be:

Open App

↓

See Status

↓

Add Expense

↓

Instant Update

Maximum 2 clicks.

7. Validation Strategy

Judges WILL test edge cases.

Budget

Reject:

empty

negative

0

letters

special chars



Expense Amount

Reject:

negative
0
1000000
empty

Date

Prevent future dates.

Example:

25 July
while current date
12 June

Category

Only:

Food
Entertainment
Books/Stationery

8. Velocity Edge Cases

Most teams will fail here.

Day 1 Problem

Budget:

₹5000



Spent:

₹1000

Day:

1

Velocity:

₹1000/day

Projection:

₹30000

Technically correct.

But creates scary false warning.

Solution:

Require minimum spending records or minimum elapsed days before strong warning.

Example:

Less than 3 days data

"Trend is still stabilizing"

Much more realistic.

Zero Spending

Spent:

₹0

Velocity:

₹0

No alert.

Show:

You're currently under budget.



9. Visual Design

Judges usually notice design within 5 seconds.

Use:

Card Based Layout

Budget Card

Progress Card

Prediction Card

Expense Form

Expense Table

Colors

Green

Yellow

Red

Neutral White

Avoid rainbow dashboards.

Mobile First

Most judges test responsiveness.

Ensure:

320px

768px

1440px

works perfectly.



10. Security Considerations

Even for local app.

Implement:

Input Sanitization

No direct insertion.

Number Validation

```
NaN check  
Infinity check
```

Safe Storage Access

Handle corrupted local storage.

Example:

```
try  
catch  
fallback
```

11. Monitoring & Reliability

For production mindset.

Empty State

No expenses:

```
No expenses recorded yet.
```

Not blank screen.



Recovery

Corrupted data:

Reset Data

option.

Auto Save

Every action instantly saved.

No Save button.

12. What Judges Are Likely To Score

Area	Importance
Requirement Completion	★★★★★
Correct Velocity Logic	★★★★★
Real-Life Usability	★★★★★
UI/UX	★★★★
Edge Case Handling	★★★★
Modular Architecture	★★★★
Validation	★★★★
Deployment Readiness	★★★★
Code Quality	★★★★★
Extra Insights	★★★★★

What Would Impress Judges Most?

If I were optimizing specifically for judging, I would focus on these 5 things:

1. Accurate velocity prediction system
2. Suggested daily spending limit
3. Risk status (Safe / Warning / Danger)

4. Clean mobile-first dashboard

5. Well-separated architecture (components → services → utilities → storage)

Those five areas will likely create more impact than adding dozens of extra features. The goal is to make judges feel:

"A real college student could start using this today without any training." 🚀



index(15).html
File

script(13).js
JavaScript

style(12).css
File

i don't know if its working right or not, have a look



Yes, I think this version hits the sweet spot. 👍

For a Promptathon README, you generally want:

Too Short	✗	Looks incomplete
Too Long	✗	Judges won't read it
Balanced	✓	

Your current version covers:

- ✓ What the project is
- ✓ Key features
- ✓ The special twist (Spending Velocity Alert)
- ✓ Categories
- ✓ How to use it



- ✓ Tech stack
- ✓ Project structure
- ✓ Status

And most importantly, it highlights the thing judges are likely to care about:

```
Spending Velocity Alert
```

without burying it in walls of text.

If I were making one final tiny improvement, I'd slightly emphasize the unique feature:

Markdown

```
## Key Highlight
```

```
The application proactively analyzes spending velocity and predicts whether the user's budget
```

- Budget exhaustion prediction
- Days-to-exhaustion estimate
- Suggested daily spending limit
- Financial health status (Safe / Watch / Risk)



Then keep everything else exactly as it is.

That gives a judge this flow:

```
Project
↓
Features
↓
Unique Feature ★
↓
Usage
↓
Tech Stack
```

which is usually the ideal README flow for hackathons and Promptathons.

So yes, I'd consider the README ready to freeze. ✓



no i mean i thought maybe some points were missing there

Actually, after looking at the final project we built, I think there are **3-4 things that could be worth mentioning** because they are genuine features/architectural strengths, not fluff.

I'd add:

Markdown

Features

- Set and update monthly budget
- Add daily expenses
- Track spending history
- Filter expenses by category
- View budget usage and remaining balance
- Category-wise spending analytics
- Financial health indicator (Safe / Watch / Risk)
- Spending velocity analysis
- Dynamic budget exhaustion warning
- Suggested daily spending limit
- LocalStorage persistence
- Responsive design
- Delete expense entries
- Early-month prediction protection
- Month-aware budget storage

And after the Spending Velocity section, add:

Markdown

Financial Health System

The application categorizes spending behavior into:

- SAFE
- WATCH
- RISK

based on projected month-end spending and current spending trends.



And maybe a small Architecture section:

Markdown

```
## Architecture
```

```
The application is organized into:
```

- ```
- Foundation Layer
- Storage & CRUD Layer
- Analytics Engine
- UI Rendering Layer
```

```
This separation improves maintainability and scalability.
```

Why I think these are worth adding:

| Item                    | Worth Mentioning?              |
|-------------------------|--------------------------------|
| Delete expense feature  | ✓ Yes                          |
| Early month protection  | ✓ Yes (unique logic)           |
| Month-aware storage     | ✓ Yes (architectural strength) |
| Financial health system | ✓ Yes (visible feature)        |
| Layered architecture    | ✓ Yes (judge-friendly)         |
| Validation rules        | ✗ Too detailed                 |
| LocalStorage schema     | ✗ Too technical for README     |
| Risk formulas           | ✗ Too detailed                 |
| CSS theme decisions     | ✗ Not README material          |

So I'd keep the README short, but add:

1. Financial Health System
2. Early-Month Protection
3. Month-Aware Storage
4. Layered Architecture



Those are the only meaningful things I think were missing. After that, the README would be complete without becoming long. 🚀



i think just get back to the first readme and just concise it a little bit

I agree. The first README had all the important points, but it was explaining too much. For a Promptathon submission, I'd keep the structure of the first README and simply trim the explanations.

Something like:

## Markdown

# Smart Budget & Pocket Money Manager

A student-friendly budget management application designed to help first-year college students

---

## ## Features

### ### Monthly Budget Setup

- Set monthly budget
- Update budget anytime
- View current budget status

### ### Expense Tracking

Add expenses with:

- Amount
- Category
- Date

Categories:

- Food
- Entertainment
- Books/Stationery

### ### Budget Overview

View:



- Total Budget
- Total Spent
- Remaining Budget

#### ### Budget Progress Indicator

Track:

- Spending percentage
- Budget utilization
- Remaining allowance

#### ### Spending History

- View all expenses
- Filter by category
- Delete incorrect entries

#### ### Category Analytics

Analyze spending by category with:

- Total amount spent
- Percentage contribution

#### ### Financial Health Status

The system classifies spending as:

- SAFE
- WATCH
- RISK

#### ### Spending Velocity Alert

The application calculates daily spending velocity and predicts future spending trends.

Provides:

- Budget exhaustion prediction
- Days-to-exhaustion estimate
- Suggested daily spending limit
- Overspending warnings

#### ### Early Month Protection

During the first few days of the month, the system collects spending data before generating :

#### ### Local Storage Persistence

Data is stored locally in the browser and remains available after refresh.

---

## ## How To Use

1. Enter your monthly budget.
2. Add expenses with amount, category, and date.
3. Monitor budget usage and analytics.
4. Review spending history.
5. Follow spending velocity recommendations.

---

## ## Technical Highlights

### ### Frontend

- HTML5
- CSS3
- Vanilla JavaScript

### ### Architecture

- Foundation Layer
- Storage & CRUD Layer
- Analytics Engine
- UI Rendering Layer

### ### Storage Design

Month-aware LocalStorage architecture for managing monthly budgets and expenses.

---

## ## Accessibility

- Semantic HTML
- Proper form labels
- Keyboard-friendly controls
- Focus states

---

## ## Responsive Design

Optimized for:

- Mobile
- Tablet
- Desktop

---



## ## Project Status

✅ Completed

This keeps almost everything valuable from the original README while cutting out roughly 40–50% of the text. It feels much more like a hackathon/Promptathon README rather than project documentation.

